OBJECT-ORIENTED ANALYSIS AND DESIGN

Assignment

Submitted By:

Rohit Sharma Pandey

M.E Computer 1st Semester

Submitted in Partial Fulfillment of the Requirement of Master's Degree in ME Computer

Submitted to:

Everest Engineering College

Sanepa, Lalitpur-2



September 2026

Table of Contents

1. Introduction 4
2. OOAD Principles 5
2.1 Encapsulation 5
House Rent Example 6
2.2 Abstraction 7
House Rent Example 7
2.3 Inheritance 7
Benefits of Inheritance 7
Important Terms 7
House Rent Example 8
2.4 Polymorphism 8
House Rent Example 8
3. Analysis Object Models and Dynamic Models 9
3.1 Entity Objects 9
3.2 Boundary Objects 9
3.3 Control Objects 9
4. Generalization and Specialization 9
4.1 Generalization 10
House Rent Example 10
4.2 Specialization 10
House Rent Example 10
5. UML Diagrams 11
5.1 Why Model Software 11
5.2 What is UML 11
5.3 Use-Case Relationships 11
<<extend>> 11
<<include>> 11
5.4 Actor 11
6. Class Diagrams 11
6.1 Composition 12
6.2 Aggregation 12
6.3 Association and Multiplicity 12
Qualifier 12
6.4 Inheritance 13
6.5 Relationship Strength 13

6.6 Complete Object Model — House Rental System 14
7. Sequence Diagrams 14
7.1 Example: Room Search and Reservation 14
7.2 Example: End-to-End House Rental Flow 15
8. Major UML Diagrams 16
9. Use Case Diagram 16
10. Activity Diagram 17
11. State Machine Diagram 18
12. Conclusion 19

List of Figures

Fig. 2.1(a): Objects encapsulate data and member functions and interact with each other through their public interfaces. 5
Fig. 2.1(b): Access levels — public members are visible everywhere, protected members are visible to derived classes, and private members are hidden from the outside world. 6
Fig. 2.1(c): UML class diagram of House showing private attributes and public accessor methods. 6
Fig. 2.3: Owner, Tenant, and Admin inherit common features from User. 8
Fig. 3.1: Actors interact through boundary objects, which delegate to control objects that coordinate the entity objects. 9
Fig. 4.1: Owner, Tenant, and Admin are generalized under the common superclass User. 10
Fig. 6.1: Composition — a Room cannot exist independently of the House it belongs to. 12
Fig. 6.2: Aggregation — Furniture can exist independently of the House. 12
Fig. 6.3: An unqualified Owner–House association (0..*) is refined into a qualified association using HouseID, reducing the multiplicity to 0..1. 13
Fig. 6.4: Inheritance hierarchy rooted at User. 13
Fig. 6.5: Composition is the strongest relationship, followed by aggregation, then plain association. 13
Fig. 6.6: Object model showing Owner, Rental (with its HouseRental / RoomRental / ApartmentRental specialisations), and Tenant. 14
Fig. 7.1: Sequence diagram for room search and reservation, including an alternative fragment for the ‘if available’ path. 15
Fig. 7.2: Sequence diagram for the House Rental System, from house search through booking confirmation to payment receipt. 16
Fig. 9.1: Use case diagram for the House Rental System. 17
Fig. 10.1: Activity diagram for the House Rental System, showing decision points and a fork/join for parallel activities. 18
Fig. 11.1: State machine diagram for the House object in the rental system. 19

1. Introduction

Object-Oriented Analysis and Design (OOAD) is the process of analysing a problem domain and designing a software solution using the concepts of objects, classes, and their interactions. This assignment applies the core principles of OOAD — encapsulation, abstraction, inheritance, and polymorphism — together with the Unified Modeling Language (UML) to model a House Rental System. The House Rental System allows Tenants to search for and book properties, Owners to list and manage properties, and an Admin to oversee users, bookings, and payments. Each concept discussed below is illustrated with a diagram, a short explanation, and a corresponding Java code snippet drawn from this case study.

2. OOAD Principles

2.1 Encapsulation

Encapsulation is the mechanism of binding data and the functions that operate on that data into a single unit — a class. It is enforced in object-oriented programming through objects and classes, which bundle an object's internal state together with the operations that are allowed to modify it.

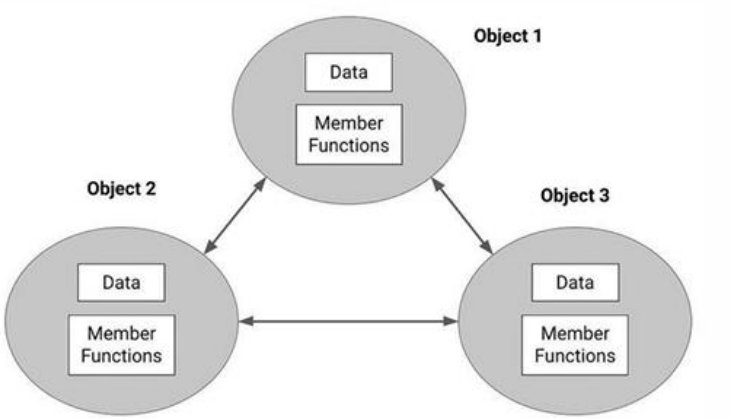


Fig. 2.1(a): Objects encapsulate data and member functions and interact with each other through their public interfaces.

Access modifiers control the visibility of a class's members and are central to enforcing encapsulation:

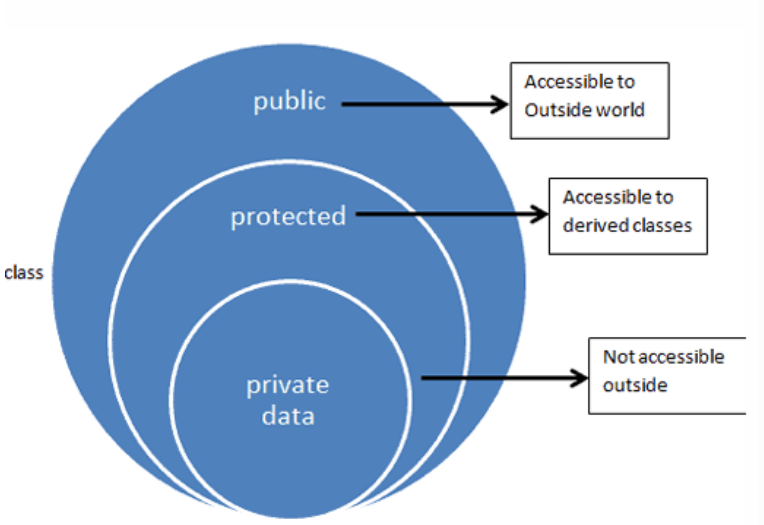


Fig. 2.1(b): Access levels — public members are visible everywhere, protected members are visible to derived classes, and private members are hidden from the outside world.

- public — accessible to the outside world.
- protected — accessible only to derived (child) classes.
- private — not accessible from outside the class at all.

House Rent Example

In the House Rental System, the House class keeps its rent, address, and status fields private, and exposes only the methods needed to read or update them (getRent(), setRent(), getStatus()). This protects the internal state of a House object from being changed directly or inconsistently by other parts of the system.

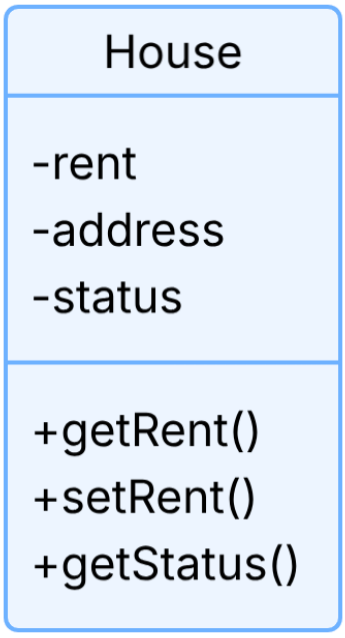


Fig. 2.1(c): UML class diagram of House showing private attributes and public accessor methods.

```
class House {
    private double rent;
```

```
    private String address;
    private boolean status;

    void setRent(double rent) {
        this.rent = rent;
    }

    double getRent() {
        return rent;
    }

    boolean getStatus() {
        return status;
    }
}
```

2.2 Abstraction

Abstraction refers to the ability to hide the internal complexity of an operation and expose only the features that are necessary to the user of a class. It provides a clear separation between what an object does and how it achieves its functionality, letting the rest of the system depend only on a stable, simplified interface.

House Rent Example

In the House Rental System, the abstract User class declares a common login() operation without specifying how each type of user actually authenticates. Every concrete subclass is then responsible for supplying its own implementation.

```
abstract class User {
    abstract void login();
}
```

2.3 Inheritance

Inheritance is a mechanism that allows classes to be organised into hierarchies, so that properties and methods can be shared between a general superclass and one or more specialised subclasses. A derived (child) class inherits the characteristics of its base (parent) class and can extend that functionality by adding new features or by overriding existing ones.

Benefits of Inheritance

- Facilitates the reuse of code by allowing a subclass to inherit attributes and methods from its superclass.
- Modifications and updates can be made once, at the superclass level, and they automatically affect all derived classes.
- Ensures consistency across related classes and simplifies ongoing maintenance.
- Changes are centralised and propagate naturally through the inheritance hierarchy.

Important Terms

- Specialisation — the act of defining one class as a refinement of another.
- Subclass — a class defined as a specialisation of a superclass using inheritance.
- Superclass — a class that serves as the base for inheritance in a class hierarchy.

House Rent Example

Owner, Tenant, and Admin each inherit the common userId attribute and login() behaviour from a shared User superclass.

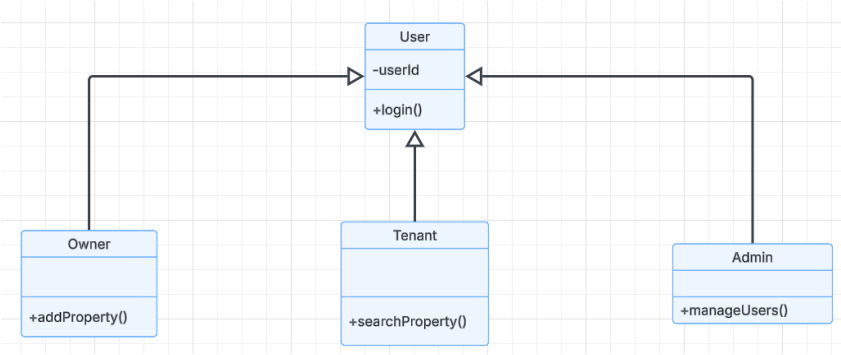


Fig. 2.3: Owner, Tenant, and Admin inherit common features from User.

```
class User {
    void login() {
        System.out.println("Login");
    }
}

class Owner extends User {
    void addProperty() {}
}

class Tenant extends User {
    void searchProperty() {}
}

class Admin extends User {
    void manageUsers() {}
}
```

2.4 Polymorphism

Polymorphism literally means “many forms.” It allows an entity to exist in multiple forms and enables different classes to be used through the same interface, with each class supplying its own implementation. As a result, objects of different classes respond to the same message in different ways.

House Rent Example

The User class defines a common login() method, but Owner, Tenant, and Admin each override it with their own implementation:

- Owner.login() → authenticates an owner.
- Tenant.login() → authenticates a tenant.
- Admin.login() → authenticates an administrator.

Because each subclass provides its own version of login(), objects of different classes respond differently to the identical login() call — this is polymorphism in action.

3. Analysis Object Models and Dynamic Models

Following the approach of Jacobson et al. (1999), the analysis object model classifies the objects of a system into three categories: entity, boundary, and control objects. Together they separate persistent data, user interaction, and use-case logic into distinct, loosely coupled responsibilities.

3.1 Entity Objects

Entity objects represent the persistent information or data tracked by the system. Examples in the House Rental System include:

- House — stores house/property information.
- Tenant — stores tenant information.
- Owner — stores owner information.
- RentPayment — stores rent payment information.

3.2 Boundary Objects

Boundary objects represent the interaction between actors and the system. Examples include:

- LoginPage — allows users to log in.
- HouseSearchPage — allows tenants to search for houses.
- PaymentPage — allows tenants to make rent payments.

3.3 Control Objects

Control objects are responsible for realising use cases and for coordinating the flow of information between boundary and entity objects. Examples include:

- LoginController — handles the login process.
- HouseSearchController — handles house-search requests.
- RentPaymentController — processes rent payments.

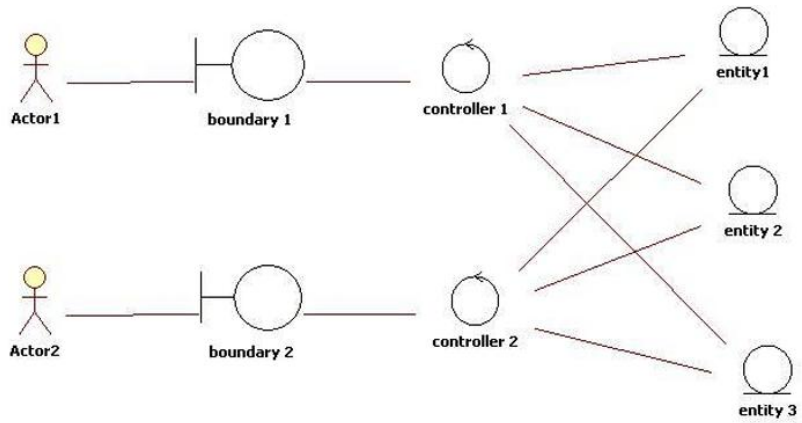


Fig. 3.1: Actors interact through boundary objects, which delegate to control objects that coordinate the entity objects.

4. Generalization and Specialization

4.1 Generalization

- Generalization is the modelling activity that identifies abstract concepts from lower-level, more specific ones.
- It represents a relationship between a more general class (the superclass) and one or more specific classes (the subclasses).
- The substitutability principle states that a more specific thing can be substituted anywhere the more general thing is expected.
- Generalization represents an “is-a” relationship.

House Rent Example

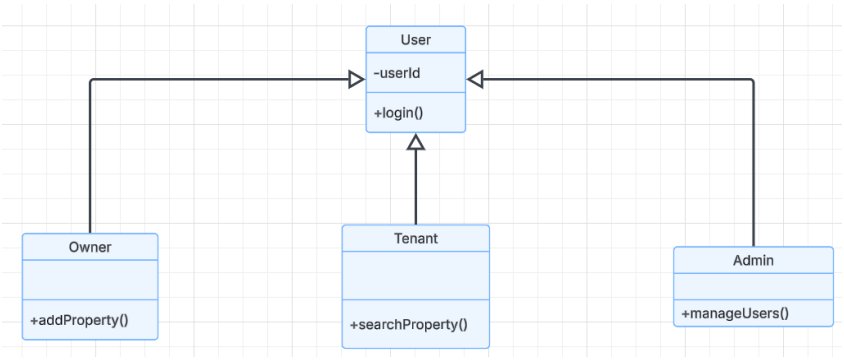


Fig. 4.1: Owner, Tenant, and Admin are generalized under the common superclass User.

Owner is a User. Tenant is a User. Admin is a User.

```
class User {
   void login() {
      System.out.println("Login");
   }
}

class Owner extends User {
   void addProperty() {}
}

class Tenant extends User {
   void searchProperty() {}
}

class Admin extends User {
   void manageUsers() {}
}
```

Here, the relationship User → Owner / Tenant / Admin represents generalization.

4.2 Specialization

- Specialization is the modelling activity that identifies specific concepts from higher-level, more general ones.
- It represents an “is-a-kind-of” relationship — the inverse perspective of generalization.

House Rent Example

Starting from the general User class, each subclass adds behaviour that specialises it for a particular role:

- Owner is a specialised type of User that adds addProperty().
- Tenant is a specialised type of User that adds searchProperty().
- Admin is a specialised type of User that adds manageUsers().

Creating Owner, Tenant, and Admin from the general User class is therefore an example of specialization.

5. UML Diagrams

5.1 Why Model Software

- Software is becoming increasingly complex and can contain millions or billions of lines of code.
- A single programmer cannot manage an entire system on their own, and code that is not modelled can be difficult for other developers to understand.
- Modelling is a means of managing this complexity by providing a shared, visual abstraction of the system.

5.2 What is UML

- UML (Unified Modeling Language) is a standard language for specifying, visualising, constructing, and documenting software systems.
- It relies mainly on graphical notations to express software design.
- UML helps development teams communicate, explore alternative designs, and validate software architecture before implementation.

5.3 Use-Case Relationships

*<<**extend**>>*

- Represents exceptional or seldom-invoked cases.
- Exceptional event flows are factored out of the main event flow for clarity.
- The direction of the <<extend>> arrow points toward the extended (base) use case.

*<<**include**>>*

- Represents behaviour that is factored out for reuse, not because it is an exception.
- It allows common behaviour to be shared by multiple use cases.
- The direction of the <<include>> arrow points toward the used (common) use case.

5.4 Actor

- An actor is an external entity that communicates with the system: a user, an external system, or the physical environment.
- Each actor has a unique name and an optional description.

Example: Tenant → Search House, Book House, Pay Rent.

6. Class Diagrams

A class diagram expresses the class definitions to be implemented in the system. It lists the name, attributes, and methods of each class, and shows the relationships between classes.

- It represents the static structure of the system.

- The class name is mandatory; attributes and methods can be shown at different levels of detail.
- A class encapsulates state (attributes) and behaviour (operations).
- Each attribute has a type, and each operation has a signature.

6.1 Composition

Composition represents a part-of / whole-part relationship in which the part cannot exist independently of the whole.



Fig. 6.1: Composition — a Room cannot exist independently of the House it belongs to.

6.2 Aggregation

Aggregation represents a consist-of / has-a relationship in which the part can exist independently of the whole.



Fig. 6.2: Aggregation — Furniture can exist independently of the House.

6.3 Association and Multiplicity

Association denotes a general relationship between classes. Multiplicity specifies how many objects of one class can participate in the association with an object of the other class.

Qualifier

A qualifier can be used to reduce the multiplicity of an association. It is used in a UML association when an object can be identified or selected using a specific key. For example, an Owner owns many House objects, but any particular house can be selected using its HouseID.

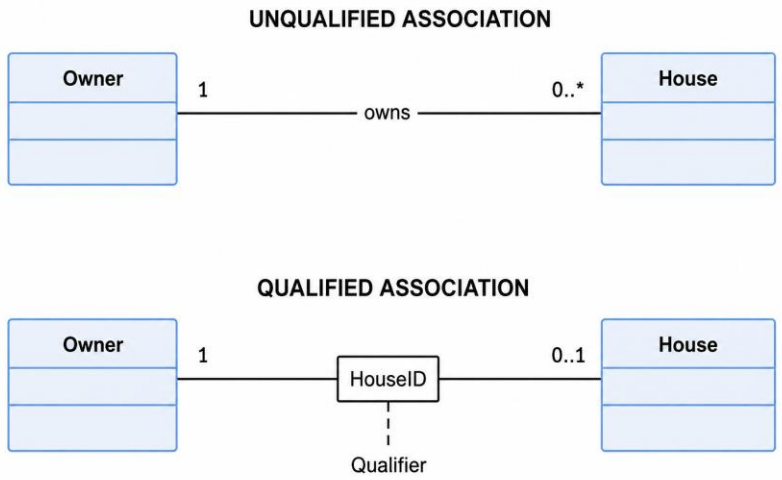


Fig. 6.3: An unqualified Owner–House association (0..) is refined into a qualified association using HouseID, reducing the multiplicity to 0..1.*

6.4 Inheritance

Inheritance is the mechanism that allows classes to be organised into hierarchies, sharing properties and methods among them, as shown earlier: Owner, Tenant, and Admin inherit common features from User.

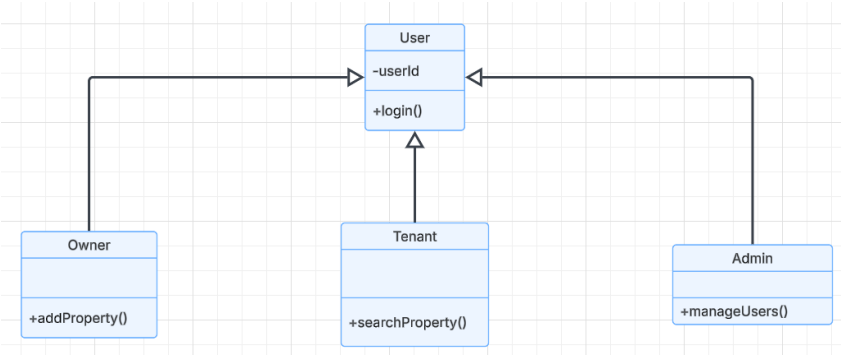


Fig. 6.4: Inheritance hierarchy rooted at User.

6.5 Relationship Strength

Association, aggregation, and composition form a spectrum of increasingly strong structural relationships between classes:

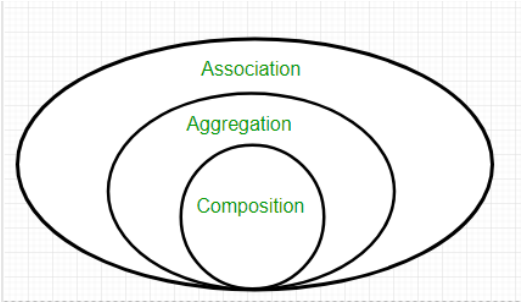


Fig. 6.5: Composition is the strongest relationship, followed by aggregation, then plain association.

6.6 Complete Object Model — House Rental System

Bringing these relationships together, the diagram below models the core rental domain: an Owner aggregates (has) many Rental records, a Tenant is associated with the Rentals they hold, and Rental is generalised into three specialised subclasses — HouseRental, RoomRental, and ApartmentRental — each adding its own attributes while sharing the common operations defined in Rental.

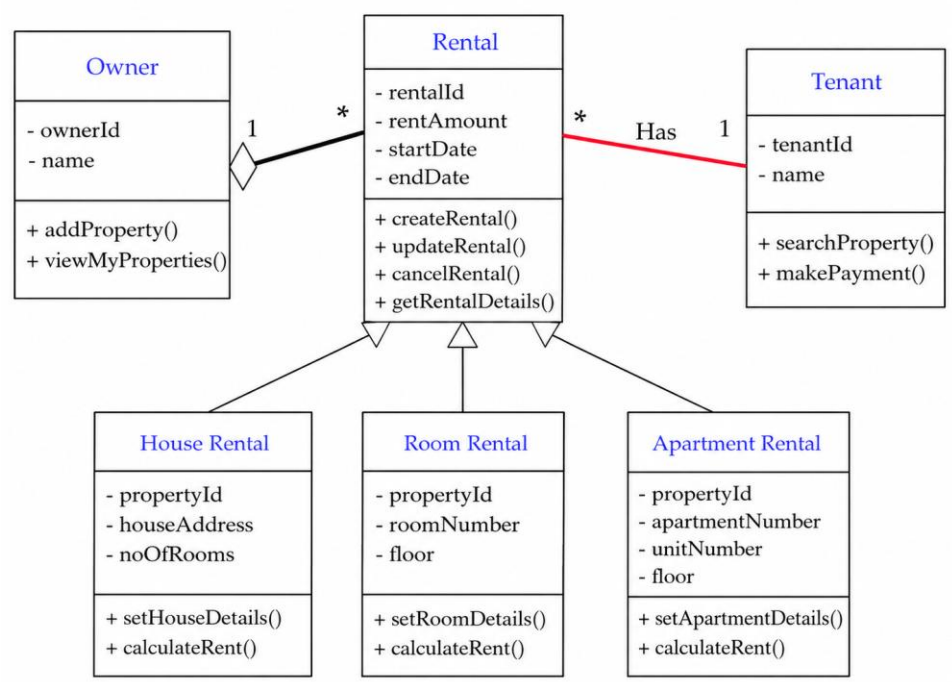


Fig. 6.6: Object model showing Owner, Rental (with its HouseRental / RoomRental / ApartmentRental specialisations), and Tenant.

7. Sequence Diagrams

A sequence diagram represents system behaviour as a set of interactions between participating objects, ordered in time. It shows:

- Objects, arranged horizontally.
- Time, flowing vertically down the page.
- Messages, shown using arrows (solid for calls, dashed for returns).
- Activations, shown using narrow rectangles on a lifeline.
- Lifelines, shown using vertical dashed lines.
- Actors, representing the external user initiating the interaction.

7.1 Example: Room Search and Reservation

A Customer searches for a room through the BookingSystem, which checks availability with the Room object. If the room is available, the customer can proceed to book it; the BookingSystem reserves the room and returns a confirmation.

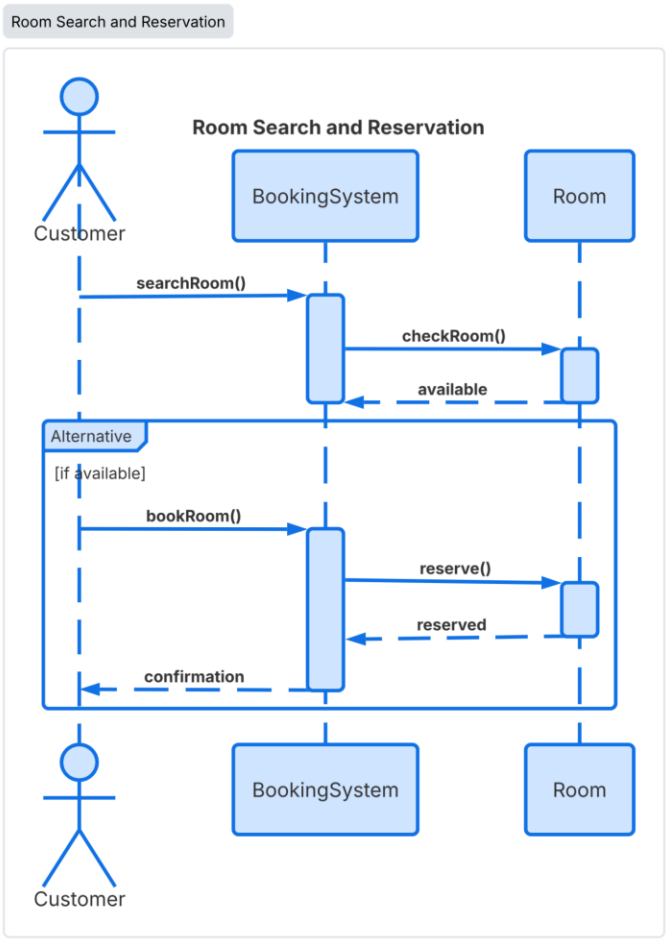


Fig. 7.1: Sequence diagram for room search and reservation, including an alternative fragment for the 'if available' path.

7.2 Example: End-to-End House Rental Flow

The diagram below traces a Tenant's complete journey through the system — searching for a house, booking it, and paying rent — across the RentalApp and its supporting services (HouseService, BookingService, PaymentService, and Time).

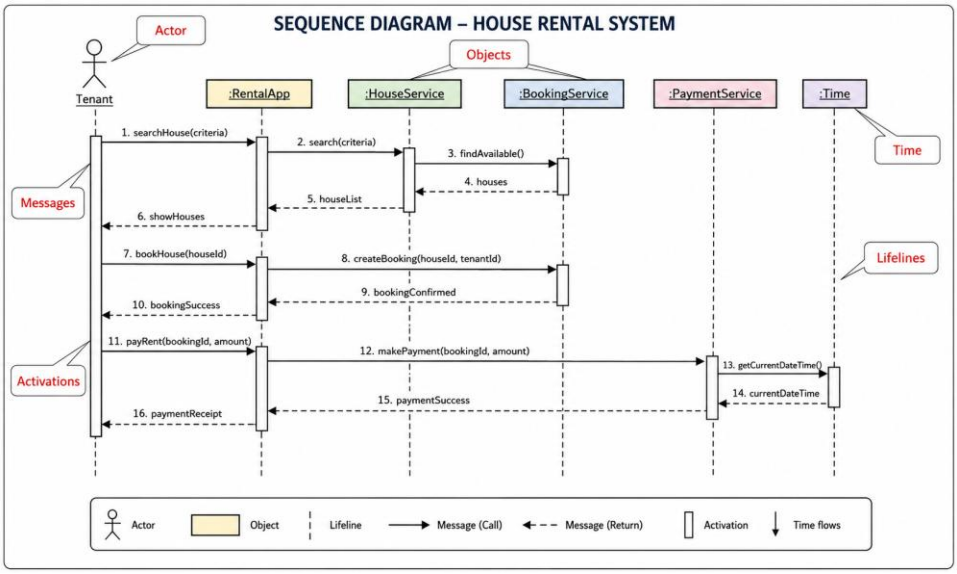


Fig. 7.2: Sequence diagram for the House Rental System, from house search through booking confirmation to payment receipt.

8. Major UML Diagrams

UML defines several complementary diagram types, each describing a different view of the system:

Diagram	Purpose
Use Case	Describes the functional requirements of the system and its interaction with external agents (actors).
Class	Describes the static structure: classes, objects, attributes, and associations.
Sequence	Describes dynamic interactions between actors and objects over time.
State Machine	Describes the dynamic behaviour of an individual object.
Activity	Describes the workflow or dynamic behaviour of a system.

9. Use Case Diagram

The use case diagram below captures the functional scope of the House Rental System from the perspective of its four actors — Tenant, Owner, Admin, and the external Payment Gateway. Include relationships factor out shared sub-behaviour (for example, both Book House and Make Payment include Process Payment where relevant), while extend relationships capture optional or exceptional behaviour such as Filter / Sort Houses or Cancel Booking.

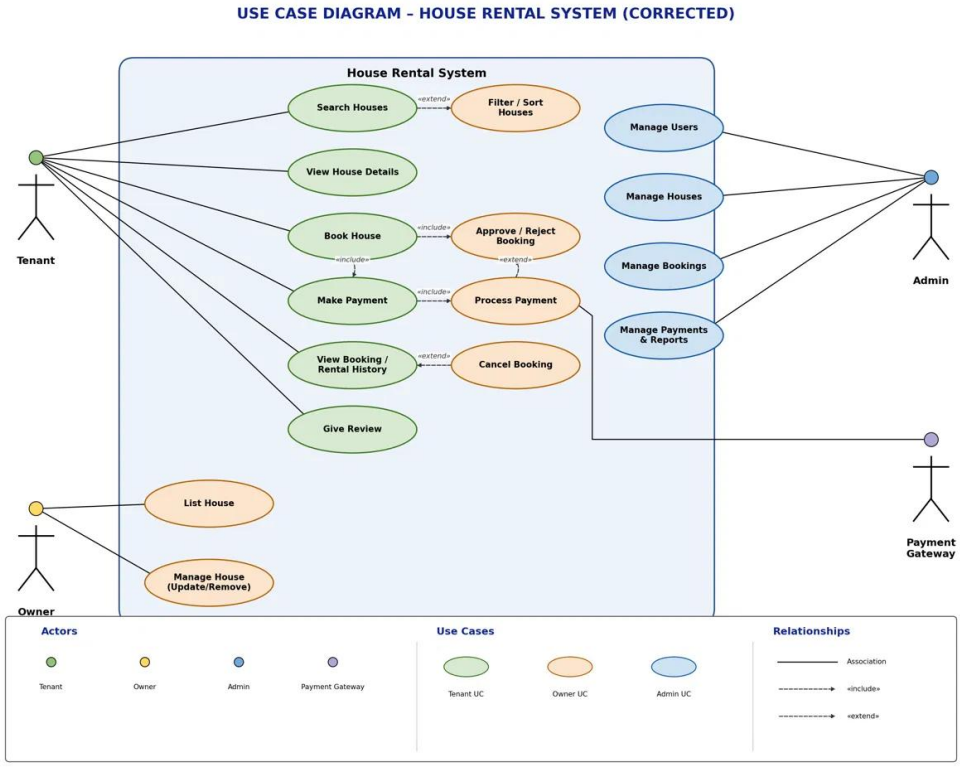


Fig. 9.1: Use case diagram for the House Rental System.

10. Activity Diagram

The activity diagram models the workflow of renting a house from start to finish. After a tenant searches for and reviews a house, a booking request is submitted for the owner's approval. Once approved, payment, document upload, and an optional site visit proceed in parallel (a fork), and are synchronised (a join) before the booking is confirmed. The rental then runs its course until the house is returned and the booking is closed.

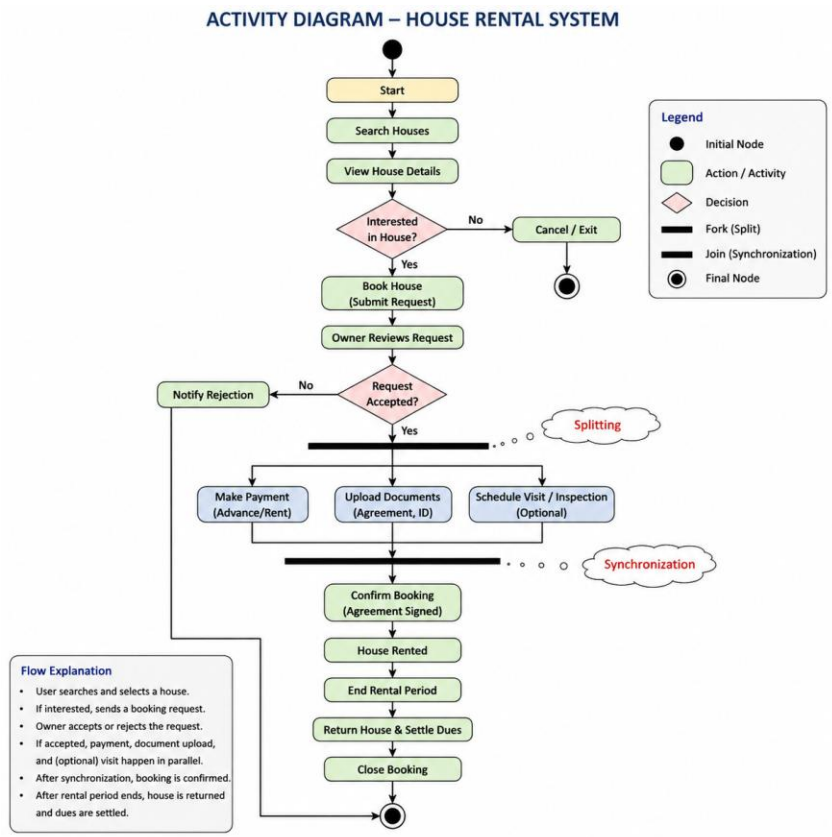


Fig. 10.1: Activity diagram for the House Rental System, showing decision points and a fork/join for parallel activities.

11. State Machine Diagram

A state machine diagram models the dynamic behaviour — the life cycle — of a single object as it responds to events. The diagram below models the House object, which moves between four states as tenants search, reserve, and rent properties, and as owners relist or remove them.

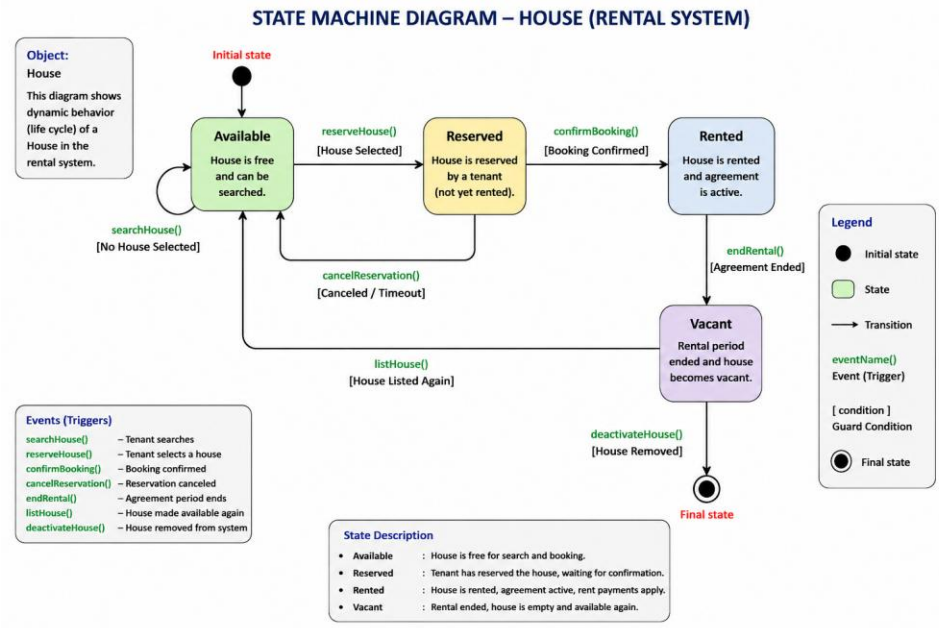


Fig. 11.1: State machine diagram for the House object in the rental system.

State	Description
Available	The house is free for search and booking.
Reserved	A tenant has reserved the house; it is awaiting confirmation.
Rented	The house is rented; the agreement is active and rent payments apply.
Vacant	The rental has ended; the house is empty and available again.

12. Conclusion

Applying OOAD principles to the House Rental System shows how encapsulation, abstraction, inheritance, and polymorphism combine to produce a maintainable class structure centred on User, Owner, Tenant, and Admin, and on the Rental hierarchy of HouseRental, RoomRental, and ApartmentRental. Layering the design into entity, boundary, and control objects keeps persistent data, user interaction, and use-case logic cleanly separated. Finally, the complementary UML views — use case, class, sequence, activity, and state machine diagrams — together give a complete picture of the system: what it does, how its data is structured, how its objects interact over time, and how individual objects behave across their life cycle.